

Documentación del Módulo de Cálculo de Resistencias Eléctricas

1. Resumen General (Overview)

El presente módulo proporciona una clase en Python denominada Resistencia, diseñada para facilitar el cálculo y la simulación de parámetros básicos en circuitos eléctricos simples. Esta versión utiliza arreglos numéricos (`numpy.ndarray`) en lugar de listas nativas de Python para la manipulación y generación de datos, optimizando el rendimiento y cambiando las estructuras internas de datos.

2. Estructura de Clases y Métodos

Clase: Resistencia

Representa el modelo y las operaciones para el trabajo con resistencias eléctricas utilizando arreglos de NumPy.

Atributos:

- `colores (np.ndarray)`: Arreglo NumPy unidimensional de cadenas de texto con los diez colores del código de resistencias estándar asociadas a los dígitos del 0 al 9 (Negro, Marrón, Rojo, Naranja, Amarillo, Verde, Azul, Violeta, Gris, Blanco).

Métodos:

Método	Parámetros	Retorno	Descripción
<code>__init__()</code>	None	None	Inicializa la instancia convirtiendo la colección estándar de colores en un arreglo NumPy (<code>np.array</code>).
<code>determinar_colores(valor)</code>	<code>valor (int str)</code> : Valor del ohmiaje.	<code>np.ndarray</code>	Calcula y retorna un arreglo con las tres primeras bandas de color asociadas al valor numérico dado.

Método	Parámetros	Retorno	Descripción
calcular_serie(n)	n (<i>int</i>): Cantidad de resistencias.	<i>tuple</i> [<i>np.ndarray</i> , <i>int</i>]	Genera un arreglo de n resistencias aleatorias usando NumPy y calcula la resistencia total equivalente en serie
calcular_paralelo(n)	n (<i>int</i>): Cantidad de resistencias.	<i>tuple</i> [<i>np.ndarray</i> , <i>float</i>]	Genera un arreglo de n resistencias aleatorias usando NumPy y calcula la sumatoria de conductancias equivalentes en paralelo

3. Documentación del Código Fuente (Docstrings Python PEP 257)

A continuación se presenta el código:

```
import random
import numpy as np
from typing import Tuple, Union

class Resistencia:
    """
    Clase para realizar cálculos y simulaciones sobre resistencias
    eléctricas utilizando arreglos NumPy.

    Attributes:
        colores (np.ndarray): Arreglo NumPy con los nombres de los
        colores correspondientes a los dígitos 0-9 del código de colores.
    """

    def __init__( -> None:
        """Inicializa los atributos de la clase Resistencia creando un
        arreglo de colores."""
        self.colores: np.ndarray = np.array([
            "Negro",      # 0
            "Marrón",     # 1
            "Rojo",       # 2
```

```

        "Naranja", # 3
        "Amarillo", # 4
        "Verde", # 5
        "Azul", # 6
        "Violeta", # 7
        "Gris", # 8
        "Blanco" # 9
    ])

    def determinar_colores(self, valor: Union[int, str]) ->
np.ndarray:
    """
        Determina las tres bandas principales de color para un valor
        dado de resistencia.

    Args:
        valor (Union[int, str]): Valor óhmico de la resistencia.

    Returns:
        np.ndarray: Arreglo NumPy de 3 elementos con los nombres
        de las bandas de color.
    """
    str_val = str(valor)
    banda1 = self.colores[int(str_val[0])]
    banda2 = self.colores[int(str_val[1])]
    banda3 = self.colores[len(str_val) - 2]
    return np.array([banda1, banda2, banda3])

    def calcular_serie(self, n: int) -> Tuple[np.ndarray, int]:
    """
        Genera un arreglo de n resistencias aleatorias y calcula su
        resistencia total equivalente en serie.

    Args:
        n (int): Cantidad de resistencias a generar.

    Returns:
        Tuple[np.ndarray, int]: Tupla que contiene el arreglo de
        resistencias generadas
                                y la resistencia total equivalente
        (Rts).
    """
    resistencias = np.array([random.randint(10, 1000000000) for _
    in range(n)])

    rts = 0
    for r in resistencias:
        rts += r

```

```

        return resistencias, rts

def calcular_paralelo(self, n: int) -> Tuple[np.ndarray, float]:
    """
    Genera un arreglo de n resistencias aleatorias y calcula el
    valor recíproco equivalente en paralelo.

    Args:
        n (int): Cantidad de resistencias a generar.

    Returns:
        Tuple[np.ndarray, float]: Tupla que contiene el arreglo de
        resistencias
                                   y la suma de sus recíprocos
        (1/Rtp) .
    """
    resistencias = np.array([random.randint(10, 1000000000) for _
in range(n)])

    rtp = 0
    for r in resistencias:
        rtp += 1 / r

    return resistencias, rtp

```

4. Guía de Uso del Menú Interactivo

El bloque de ejecución principal (if `__name__ == "__main__":`) implementa un menú en consola idéntico a la versión anterior, asegurando la compatibilidad de uso:

1. **Determinar colores:** Solicita al usuario un número (ej. 160) e imprime la combinación de las tres primeras bandas desde el arreglo.
2. **Resistencias en serie:** Solicita la cantidad *n* de resistencias a simular, genera un arreglo de NumPy con valores entre 10 y 1,000,000,000 Ω y devuelve la suma total.
3. **Resistencias en paralelo:** Solicita la cantidad *n* de resistencias a simular y calcula el resultado equivalente recíproco sobre los elementos del arreglo.
4. **Salir:** Finaliza el ciclo del menú.